Kopilkin

Microservices, AI Assistant, gRPC, Kafka, Worker, Tests and Documentation

Student Name

Mubarakov Islam

Group

11-314a

Assignment

3rd task series (2nd semester)

GitHub Repository

https://github.com/iz-mv/Kopilkin

Kopilkin Full Project Report

Microservices, AI Assistant, gRPC, Kafka, Worker, Tests and Documentation

Student Name	Mubarakov Islam
Group	11-314a
Assignment	3rd task series (2nd semester)
Project	Kopilkin Personal Finance Application
GitHub Repository	https://github.com/iz-mv/Kopilkin

Full Project Report

English Part

1. Project Overview

Kopilkin is a mobile-first personal finance web application implemented as a microservice-oriented system. Users can register and log in, track income and expenses, create savings goals, upload profile and goal images, ask an AI assistant for financial analysis, and receive smart recommendations. The final version of the project demonstrates not only separate FastAPI services, but also service-owned PostgreSQL databases, Redis caching and synchronization, Kafka-based event-driven processing, gRPC communication, MinIO object storage, AI observability and integration tests.

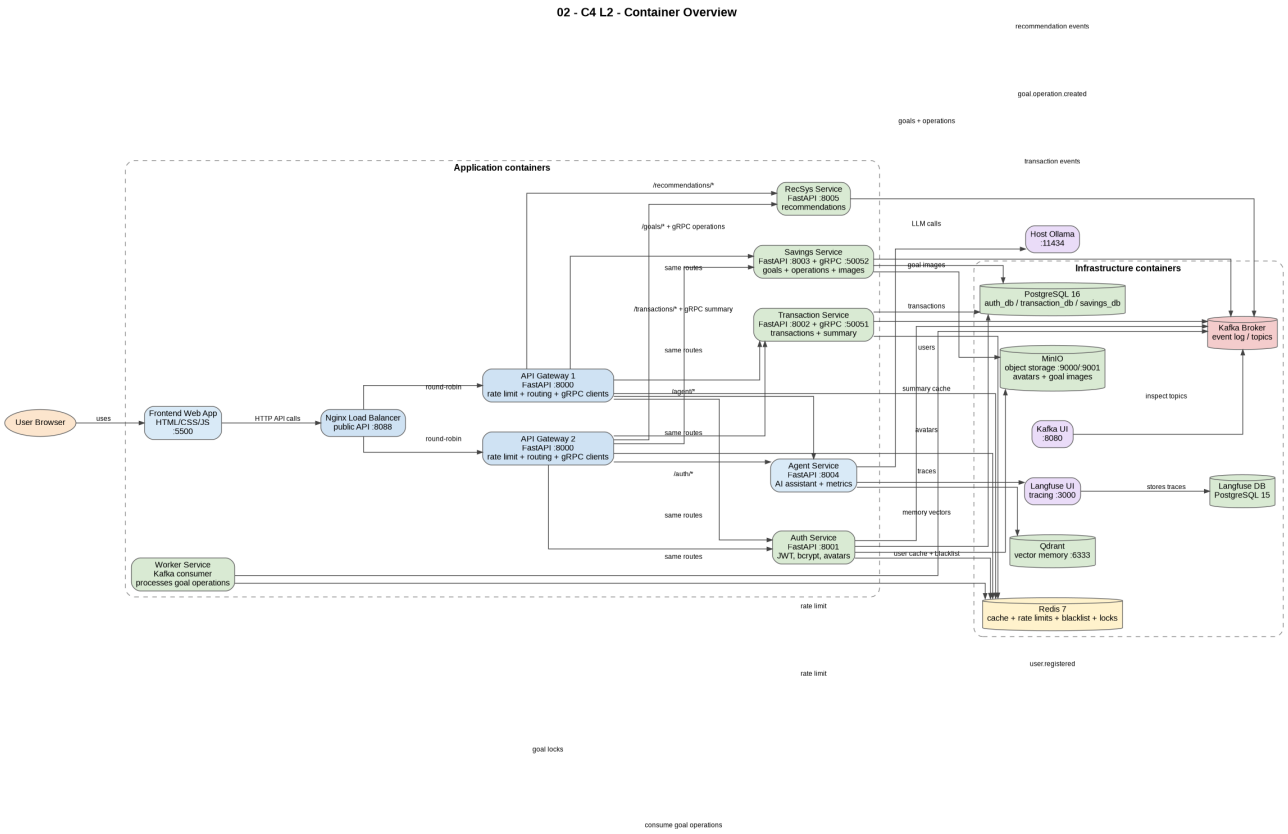

Figure 1. Final container-level architecture overview.

2. Final Scope and Requirements Mapping

Requirement / Area	Final implementation	Evidence in project
Kafka / EDA	Kafka broker, Kafka UI, domain events and a real worker consumer for goal operations.	docker-compose.yml; events.py files; worker-service/app/main.py
RDBMS + NoSQL / storage	PostgreSQL databases for business data, MinIO for files, Qdrant for AI memory, Redis for short-lived data, Kafka as event log.	infra/postgres/init.sql; service models.py; MinIO/Qdrant/Redis/Kafka containers
Redis caching / coordination	User profile cache, transaction summary cache, API Gateway rate limits, JWT blacklist, goal operation locks.	auth-service/app/cache.py; transaction-service/app/cache.py; api-gateway/app/main.py; worker-service/app/main.py
Routing layer	Frontend calls Nginx on port 8088; Nginx	infra/nginx/nginx.conf; api-

Requirement / Area	Final implementation	Evidence in project
	balances traffic to two API Gateway instances; gateways route to internal services.	gateway/app/main.py; docker-compose.yml
gRPC communication	API Gateway uses gRPC for transaction summary and goal operation creation.	proto/transaction.proto; proto/savings.proto; grpc_server.py; api-gateway/app/grpc_client.py
Worker service	Asynchronous processing of goal operations from Kafka with Redis locking and transaction mirroring.	worker-service/app/main.py; topic goal.operation.created
Object storage	User avatars and goal images are uploaded to MinIO; PostgreSQL stores only URLs.	auth-service/app/storage.py; savings-service/app/storage.py
AI assistant	Agent service uses local Ollama, Qdrant memory and Langfuse traces; it reads transaction data for financial analysis.	agent-service/app/main.py; analyst_agent.py; system_prompts.py
Recommender system	Separate RecSys microservice reads transactions and produces financial recommendations.	recsys-service/app/main.py
Testing	12 integration tests verify real Docker Compose service flows.	tests/ directory; pytest output: 12 passed

3. Microservice Architecture

The system uses a set of independently deployable FastAPI services. The frontend communicates with a single public endpoint, while the internal services communicate through REST, gRPC, Kafka and shared infrastructure components. The API Gateway hides internal service ports from the frontend and applies Redis-based rate limiting. Two gateway instances are placed behind Nginx to demonstrate load balancing.

Service	Role	Port / interface
frontend	Browser user interface	5500
nginx-balancer	Public entry point and load balancer	8088
api-gateway-1 / api-gateway-2	Routing, rate limiting, gRPC clients	internal 8000
auth-service	Users, JWT, bcrypt, Redis blacklist, avatars	8001
transaction-service	Transactions, summaries, cache, Transaction gRPC server	8002 / 50051
savings-service	Goals, images, GoalOperation, Savings gRPC server	8003 / 50052
worker-service	Kafka consumer and async operation processor	internal
agent-service	AI assistant, memory, traces, metrics	8004
recsys-service	Financial recommendations	8005
PostgreSQL / Redis / Kafka / MinIO / Qdrant / Langfuse	Infrastructure layer	various ports

4. Data Architecture

Kopilkin follows the database-per-service idea at the logical database level. Auth, transaction and savings data are separated into different PostgreSQL databases. Object files are not stored inside relational tables; MinIO stores avatars and goal images, while PostgreSQL keeps only the resulting public URLs. Redis is used where low-latency temporary data is needed, and Qdrant stores AI vector memory.

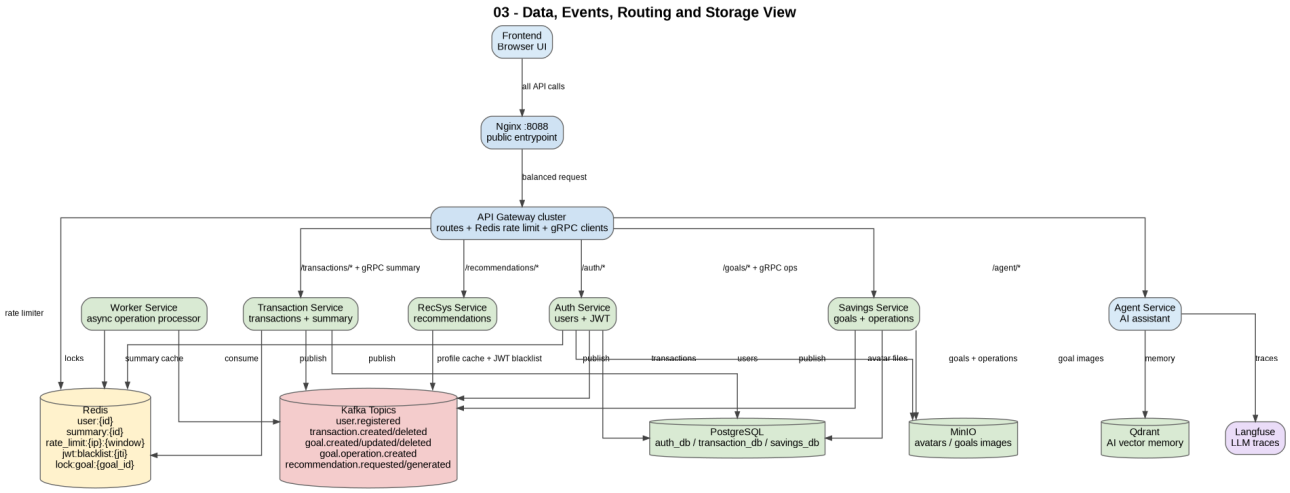

Figure 2. Data, events, routing, cache and object storage view.

Data	Storage	Owner / Reason
Users	PostgreSQL auth_db.users	Owned by auth-service; structured user data and uniqueness.
Transactions	PostgreSQL transaction_db.transactions	Owned by transaction-service; reliable financial records.
Savings goals	PostgreSQL savings_db.savings_goals	Owned by savings-service; goal state changes over time.
Goal operations	PostgreSQL savings_db.goal_operations	Created by savings-service and processed by worker-service.
Avatar files	MinIO kopilkin-files/avatars	Object storage is better for images than relational columns.
Goal images	MinIO kopilkin-files/goals	Stored as objects; service keeps only image_url.
AI memory	Qdrant collection kopilkin_memory	Vector search for semantic memory.
Temporary data	Redis	Caches, rate limits, JWT blacklist and locks.
Domain events	Kafka topics	Durable event stream for EDA and inspection.

5. Kafka and Event-Driven Processing

Kafka is used as the central event broker. Services publish domain events after successful business operations. The strongest event-driven use case is savings goal operations: the savings service creates a GoalOperation and publishes goal.operation.created; the worker-service consumes this event and updates the system asynchronously.

Topic	Producer	Consumer / Meaning
user.registered	auth-service	User account created.
transaction.created / transaction.deleted	transaction-service	Financial transaction changes; useful for analytics and recommendations.
goal.created / goal.updated / goal.deleted	savings-service	Savings goal lifecycle events.
goal.image.updated	savings-service	Goal image changed.
goal.operation.created	savings-service or Savings gRPC server	Consumed by worker-service to process deposit/withdraw.
recommendation.requested / recommendation.generated	recsys-service	Recommendation lifecycle events.

6. gRPC Communication

The final version uses gRPC in two places. First, the API Gateway calls Transaction Service through gRPC to return financial summaries. Second, the API Gateway calls Savings Service through gRPC to create deposit and withdraw operations. This makes gRPC part of a real business flow rather than a separate demonstration.

gRPC method	Caller	Server	Business purpose
GetUserSummary	API Gateway	Transaction Service :50051	Returns income, expense and balance for a user.
CreateGoalOperation	API Gateway	Savings Service :50052	Creates a pending deposit/withdraw operation and publishes Kafka event.

7. Main Business Flow: Goal Operation

The most important runtime scenario is the savings goal operation flow. A user deposits or withdraws money from a savings goal. The frontend sends an HTTP request to the gateway. The gateway calls Savings Service using gRPC. The savings service creates a GoalOperation with PENDING status and publishes a Kafka event. The worker-service consumes the event, acquires a Redis lock for the goal, changes the operation status, updates current_amount, and creates a linked transaction in the Transaction Service.

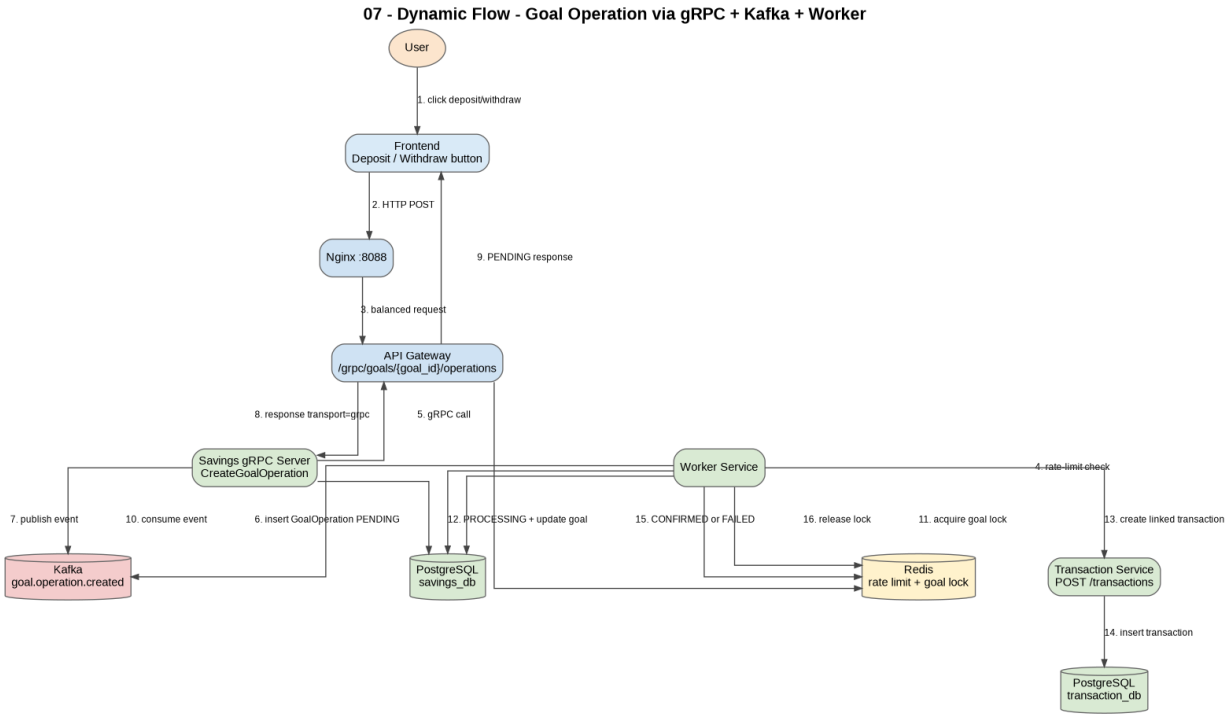

Figure 3. Dynamic goal operation flow through gRPC, Kafka and worker-service.

Status	Meaning
PENDING	Operation has been created and waits for worker processing.
PROCESSING	Worker has started processing the operation.
CONFIRMED	Operation was applied successfully and linked transaction was created.
FAILED	Operation failed, for example because withdraw amount was larger than current goal amount.
CANCELLED	Reserved for future cancellation logic.

8. Redis Usage

Redis is used in several practical places. It is not only a cache, but also part of authentication, routing and asynchronous coordination.

Use case	Key pattern	TTL / behavior
User profile cache	user:{user_id}	Short TTL; invalidated after profile update.
Transaction summary cache	summary:{user_id}	TTL 60 seconds; invalidated after transaction create/delete.
Rate limiting	rate_limit:{client_ip}:{minute_window}	Gateway increments counters and returns HTTP 429 when limit is exceeded.
JWT blacklist	jwt:blacklist:{jti}	Token jti is stored after logout until token expiration.
Goal operation lock	lock:goal:{goal_id}	Worker uses SET NX EX to avoid simultaneous updates.

9. MinIO Object Storage

MinIO is used for binary image files. The auth-service uploads user avatars to the avatars folder. The savings-service uploads goal images to goal-specific folders. Old images are deleted or cleaned up after replacement. This avoids storing image bytes in PostgreSQL and keeps database rows lightweight.

10. AI Assistant and Observability

The AI assistant is implemented in agent-service. It uses the local Ollama runtime for LLM and embeddings, Qdrant for semantic memory, and Langfuse for traces. The analyst agent reads real transaction data and builds context about total income, expenses, categories and previous-month transactions. The prompts include a language rule: if the user writes in Russian, the assistant answers in Russian; if the user writes in English, it answers in English.

11. Recommender System

RecSys is a separate FastAPI service. It reads transactions from transaction-service, generates personalized financial recommendations, and publishes recommendation events to Kafka. The current implementation combines heuristic recommendations, biggest expense category detection, educational collaborative filtering with synthetic profiles, and cold-start advice.

12. Integration Testing

The project includes integration tests that run against real Docker Compose services. They verify the system as a whole rather than isolated functions. The current expected result is 12 passed tests.

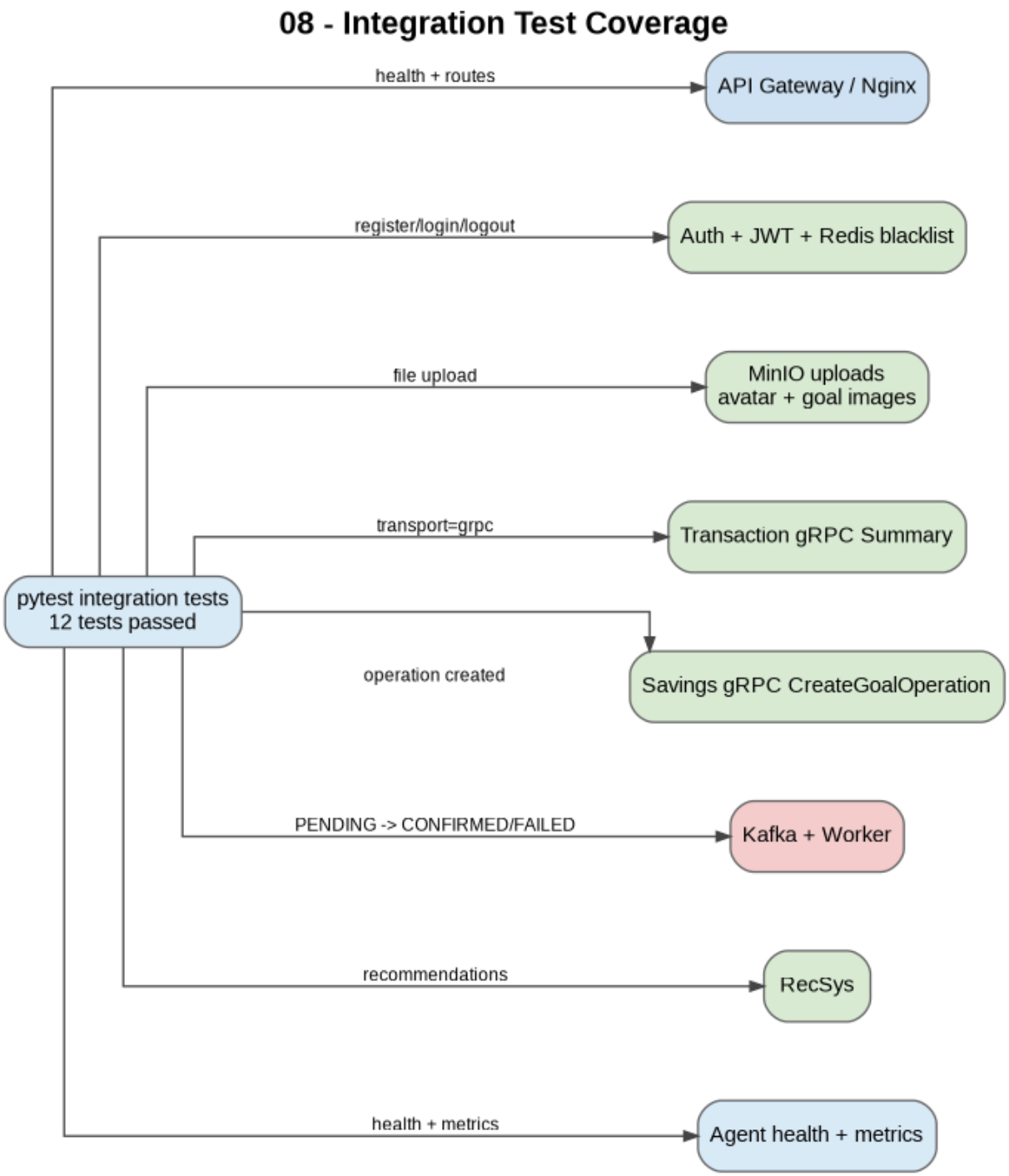

Figure 4. Integration test coverage.

Test file	What it verifies
test_01_health.py	Gateway, core service health, frontend, Kafka UI, agent health and metrics.
test_02_auth_jwt_and_minio.py	Register/login/me/logout, Redis JWT blacklist, avatar upload to MinIO.
test_03_transactions_grpc_summary.py	Transaction creation, gRPC financial summary, transaction idempotency.
test_04_savings_goal_grpc_worker.py	Goal image upload, gRPC CreateGoalOperation, Kafka worker, status transitions, failed withdraw.
test_05_recommendations.py	Recommendations generated from real transactions.

13. Conclusion

The final version of Kopilkin is a complete educational microservice project. It includes real business functionality, REST routing, gRPC communication, event-driven Kafka processing, Redis-based caching and coordination, persistent PostgreSQL storage, MinIO object storage, AI assistant integration, observability and integration tests. The project is suitable for demonstrating the main competencies required in the third task series.

Русская часть

1. Обзор проекта

Kopilkin - это mobile-first веб-приложение для управления личными финансами, реализованное как микросервисная система. Пользователь может регистрироваться и входить в аккаунт, добавлять доходы и расходы, создавать цели накоплений, загружать аватар и изображения целей, спрашивать AI-ассистента о финансах и получать рекомендации. Финальная версия показывает не только отдельные FastAPI-сервисы, но и service-owned PostgreSQL databases, Redis caching and synchronization, Kafka-based event-driven processing, gRPC communication, MinIO object storage, AI observability и integration tests.

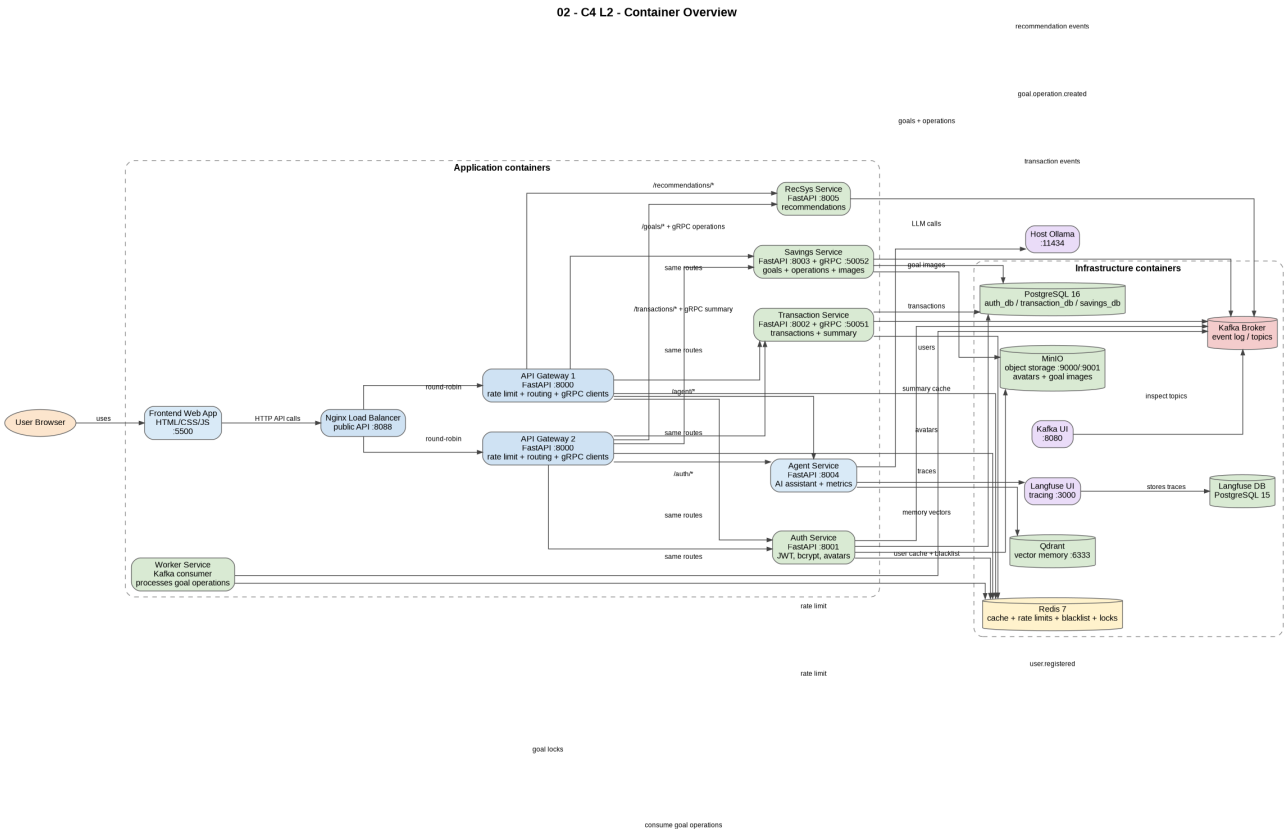

Рисунок 1. Финальная container-level архитектура.

2. Объём проекта и связь с требованиями

Требование / область	Финальная реализация	Где видно в проекте
Kafka / EDA	Kafka broker, Kafka UI, доменные события и настоящий worker consumer для операций по целям.	docker-compose.yml; events.py; worker-service/app/main.py
RDBMS + NoSQL / storage	PostgreSQL для бизнес-данных, MinIO для файлов, Qdrant для AI memory, Redis для временных данных, Kafka как event log.	infra/postgres/init.sql; models.py; контейнеры MinIO/Qdrant/Redis/Kafka
Redis caching / coordination	Кэш профиля, кэш summary, rate limits, JWT blacklist, locks для целей.	cache.py; api-gateway/app/main.py; worker-service/app/main.py
Routing layer	Frontend обращается к Nginx на порту 8088; Nginx балансирует два API	infra/nginx/nginx.conf; api-gateway/app/main.py

Требование / область	Финальная реализация	Где видно в проекте
	Gateway; gateway маршрутизирует во внутренние сервисы.	
gRPC communication	API Gateway использует gRPC для summary транзакций и создания операций по целям.	proto files; grpc_server.py; api-gateway/app/grpc_client.py
Worker service	Асинхронная обработка операций целей из Kafka с Redis lock и созданием связанной transaction.	worker-service/app/main.py; topic goal.operation.created
Object storage	Аватары и картинки целей загружаются в MinIO; в PostgreSQL хранится только URL.	storage.py в auth и savings service
AI assistant	Agent service использует Ollama, Qdrant memory и Langfuse traces; читает financial data.	agent-service; analyst_agent.py; system_prompts.py
Recommender system	Отдельный RecSys microservice читает транзакции и генерирует рекомендации.	recsys-service/app/main.py
Testing	12 integration tests проверяют реальные Docker Compose flows.	tests/; pytest output: 12 passed

3. Микросервисная архитектура

Система состоит из набора FastAPI-сервисов. Frontend работает только с публичным входом через Nginx и API Gateway. Внутри сервисы взаимодействуют через REST, gRPC, Kafka и общие инфраструктурные компоненты. API Gateway скрывает внутренние порты сервисов и применяет Redis-based rate limiting. Две копии gateway стоят за Nginx, чтобы показать load balancing.

Сервис	Роль	Порт / интерфейс
frontend	Пользовательский интерфейс	5500
nginx-balancer	Публичный вход и балансировщик	8088
api-gateway-1 / api-gateway-2	Routing, rate limiting, gRPC clients	internal 8000
auth-service	Users, JWT, bcrypt, Redis blacklist, avatars	8001
transaction-service	Transactions, summaries, cache, Transaction gRPC server	8002 / 50051
savings-service	Goals, images, GoalOperation, Savings gRPC server	8003 / 50052
worker-service	Kafka consumer and async processor	internal
agent-service	AI assistant, memory, traces, metrics	8004
recsys-service	Financial recommendations	8005
PostgreSQL / Redis / Kafka / MinIO / Qdrant / Langfuse	Infrastructure layer	разные порты

4. Архитектура данных

Kopilkin использует database-per-service подход на уровне логических баз данных. Данные пользователей, транзакций и целей разделены по PostgreSQL databases. Файлы не хранятся в таблицах: MinIO хранит аватары и картинки целей, а PostgreSQL хранит только URL. Redis используется для временных данных, Qdrant - для AI vector memory.

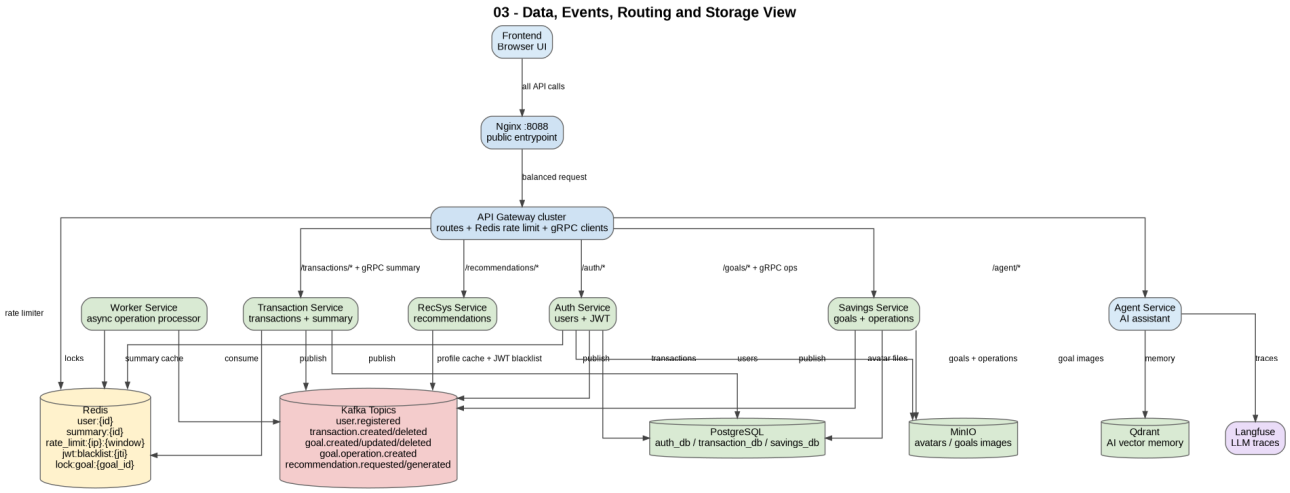

Рисунок 2. Data, events, routing, cache и object storage view.

5. Kafka и event-driven processing

Kafka используется как центральный брокер событий. Сервисы публикуют domain events после успешных бизнес-операций. Самый сильный event-driven сценарий - операции по целям накоплений: savings-service создаёт GoalOperation и публикует goal.operation.created, а worker-service читает событие и асинхронно обновляет систему.

Topic	Producer	Смысл / consumer
user.registered	auth-service	Создан новый пользователь.
transaction.created / transaction.deleted	transaction-service	Изменения финансовых транзакций; полезны для analytics и recommendations.
goal.created / goal.updated / goal.deleted	savings-service	Жизненный цикл savings goal.
goal.image.updated	savings-service	Изображение цели изменено.
goal.operation.created	savings-service или Savings gRPC server	Читается worker-service для обработки deposit/withdraw.
recommendation.requested / recommendation.generated	recsys-service	Жизненный цикл recommendations.

6. gRPC communication

Финальная версия использует gRPC в двух местах. Во-первых, API Gateway вызывает Transaction Service через gRPC для financial summary. Во-вторых, API Gateway вызывает Savings Service через gRPC для создания deposit/withdraw operations. Это делает gRPC частью реального business flow, а не отдельной демонстрацией.

gRPC method	Caller	Server	Назначение
GetUserSummary	API Gateway	Transaction Service :50051	Возвращает income, expense и balance пользователя.
CreateGoalOperation	API Gateway	Savings Service :50052	Создаёт pending deposit/withdraw operation и публикует Kafka event.

7. Главный бизнес-flow: операция по цели

Самый важный runtime-сценарий - операция по цели накопления. Пользователь пополняет цель или снимает деньги с цели. Frontend отправляет HTTP-запрос в gateway. Gateway вызывает Savings Service через gRPC. Savings Service создаёт GoalOperation со статусом PENDING и публикует Kafka event. Worker-service читает event, берёт Redis lock на goal, меняет статус operation, обновляет current_amount и создаёт связанную transaction в Transaction Service.

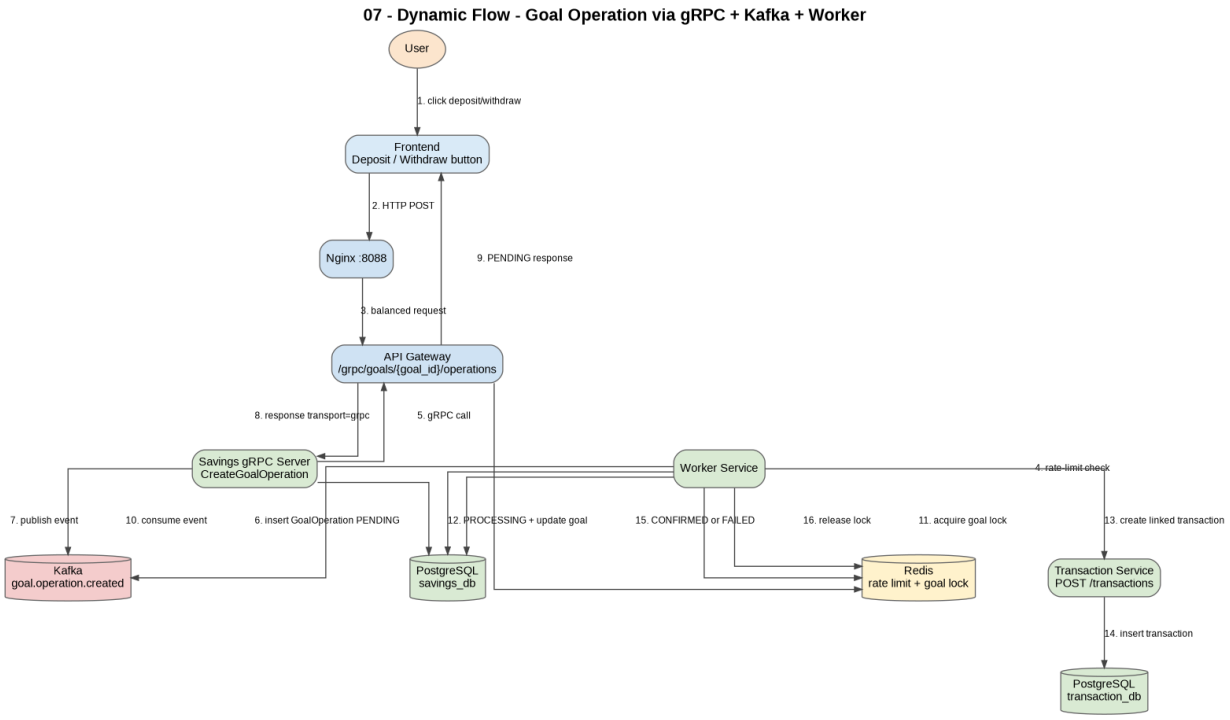

Рисунок 3. Dynamic goal operation flow через gRPC, Kafka и worker-service.

Status	Смысл
PENDING	Операция создана и ждёт обработки worker-ом.
PROCESSING	Worker начал обработку операции.
CONFIRMED	Операция успешно применена, связанная transaction создана.
FAILED	Операция не удалась, например withdraw больше текущей суммы цели.
CANCELLED	Зарезервировано для будущей логики отмены.

8. Redis usage

Redis используется в нескольких практических местах. Это не только cache, но и часть authentication, routing и asynchronous coordination.

Use case	Key pattern	TTL / поведение
User profile cache	user:{user_id}	Короткий TTL; invalidation после обновления профиля.
Transaction summary cache	summary:{user_id}	TTL 60 секунд; invalidation после create/delete transaction.
Rate limiting	rate_limit:{client_ip}:{minute_window}	Gateway увеличивает counters и возвращает HTTP 429 при превышении лимита.
JWT blacklist	jwt:blacklist:{jti}	Token jti хранится после logout до окончания срока действия token.
Goal operation lock	lock:goal:{goal_id}	Worker использует SET NX EX, чтобы избежать одновременных обновлений.

9. MinIO object storage

MinIO используется для binary image files. Auth-service загружает user avatars в папку avatars. Savings-service загружает картинки целей в goal-specific folders. Старые изображения удаляются или очищаются после замены. Это не перегружает PostgreSQL файлами.

10. AI assistant и observability

AI assistant реализован в agent-service. Он использует local Ollama runtime для LLM и embeddings, Qdrant для semantic memory и Langfuse для traces. Analyst agent читает реальные transaction data и строит контекст по доходам, расходам, категориям и активности за прошлый месяц. В prompt добавлено правило языка: если пользователь пишет на русском, ассистент отвечает на русском; если на английском - на английском.

11. Recommender System

RecSys - отдельный FastAPI service. Он читает transactions из transaction-service, генерирует personalized financial recommendations и публикует recommendation events в Kafka. Сейчас используются heuristic rules, biggest expense category detection, учебный collaborative filtering с synthetic profiles и cold-start advice.

12. Integration testing

В проект добавлены integration tests, которые запускаются против реальных Docker Compose services. Они проверяют систему целиком, а не только отдельные функции. Ожидаемый результат: 12 passed tests.

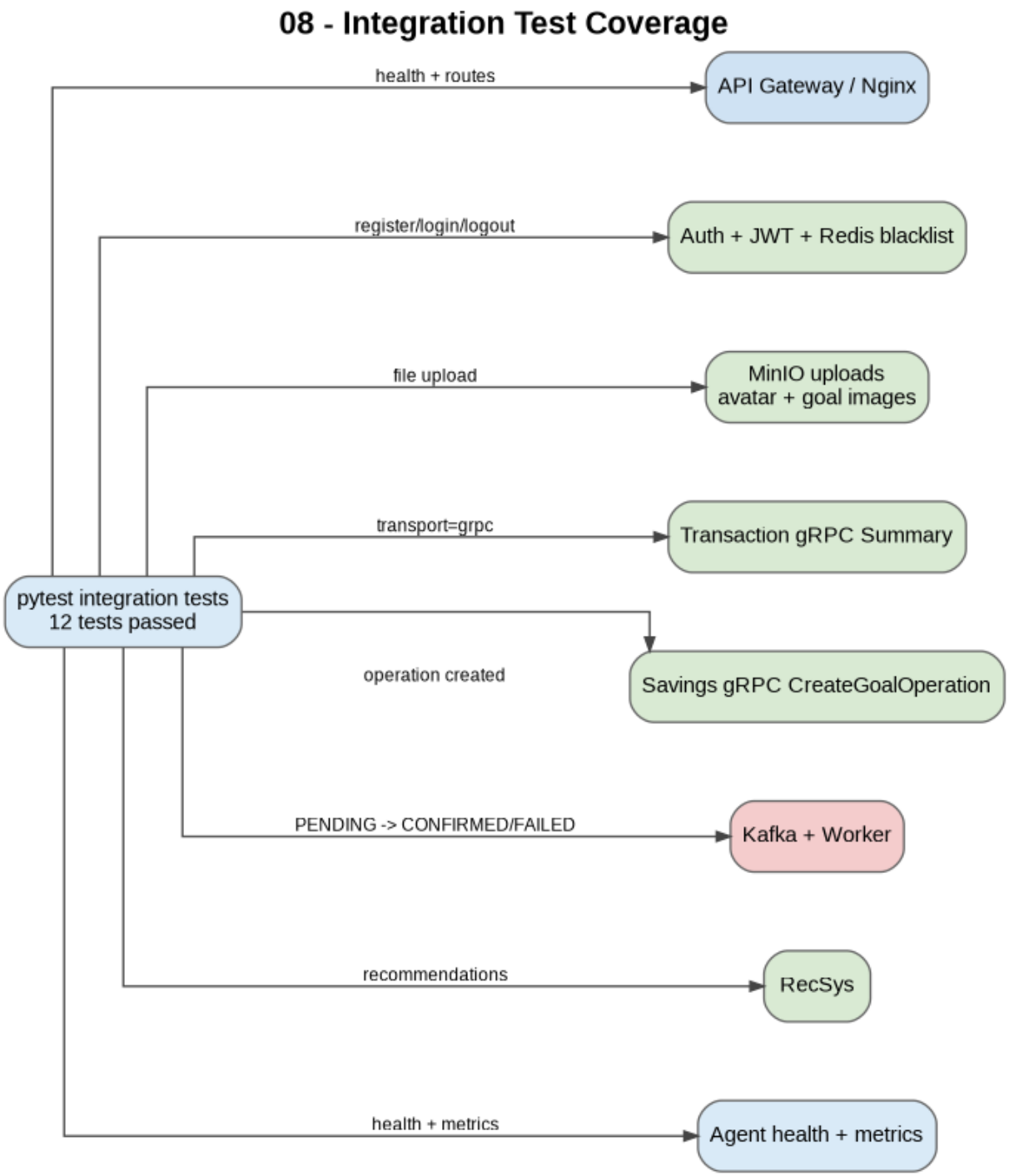

Рисунок 4. Покрытие integration tests.

Test file	Что проверяет
test_01_health.py	Gateway, health сервисов, frontend, Kafka UI, agent health and metrics.
test_02_auth_jwt_and_minio.py	Register/login/me/logout, Redis JWT blacklist, avatar upload to MinIO.
test_03_transactions_grpc_summary.py	Transaction creation, gRPC financial summary, transaction idempotency.
test_04_savings_goal_grpc_worker.py	Goal image upload, gRPC CreateGoalOperation, Kafka worker, status transitions, failed withdraw.
test_05_recommendations.py	Recommendations from real transactions.

13. Заключение

Финальная версия Kopilkin - полноценный учебный микросервисный проект. В нём есть реальные business features, REST routing, gRPC communication, event-driven Kafka processing, Redis-based caching and coordination, persistent PostgreSQL storage, MinIO object storage, AI assistant, observability и integration tests. Проект подходит для демонстрации основных компетенций третьего блока заданий.